Week 5: PHP and MySQL

5. Introduction to MySQL database and PHP integration

MySQL is an open-source relational database management system (RDBMS) that is widely used for storing and managing structured data. It's known for its speed, reliability, and ease of use, making it a popular choice for a wide range of applications, from small-scale web applications to large enterprise-level systems.

Here's an introduction to the key concepts and features of MySQL:

5.1. **Relational Database Management System (RDBMS):** A Relational Database Management System (RDBMS) is a type of database management system that stores and manages data in a structured manner using a relational model. This model organizes data into tables (also known as relations) composed of rows and columns, where each row represents a record and each column represents an attribute or field of that record. The relationships between tables are established through keys, primarily using primary and foreign keys.

Data in MySQL is organized into tables. Each table consists of rows and columns. A row represents a single record or data entry, and columns represent different attributes or fields of the data. For example, in a "Customers" table, each row might represent a different customer, with columns like "Name," "Email," and "Phone Number."

Here are some key characteristics and concepts associated with RDBMS:

- **Tables**: RDBMSs organize data into tables, which are two-dimensional structures composed of rows and columns. Each table represents a distinct entity or concept within the database.
- **Rows**: Rows, also referred to as tuples or records, represent individual instances of data stored in a table. Each row contains data values related to the attributes defined by the table's columns.
- **Columns**:Columns, also known as fields, define the different attributes of the data. Each column holds a specific type of data, such as text, numbers, dates, or binary data.
- **Primary Keys**: Each table typically has one or more columns designated as primary keys. A primary key uniquely identifies each row in a table. It ensures that each record is unique and serves as a reference point for relationships between tables.
- Foreign Keys: When a relationship exists between two tables, a foreign key is used to link them. The foreign key in one table refers to the primary key in another table, establishing a relationship and maintaining data integrity.
- Indexes: Indexes are data structures that improve the speed of data retrieval operations on a database table. They work similar to the index of a book, allowing faster lookup of data based on certain columns.

- Normalization: Normalization is the process of organizing data in a way that minimizes redundancy and dependency. It involves dividing larger tables into smaller ones and using relationships to link them, which helps maintain data integrity and reduces data duplication.
- Concurrency Control: RDBMSs support multiple users accessing the database simultaneously. Concurrency control mechanisms manage access to the data to prevent conflicts and ensure that changes made by one user do not interfere with changes made by others.

5.2. **SQL (Structured Query Language):** SQL is the language used to interact with MySQL databases. It's used for creating, retrieving, updating, and deleting data in the database. SQL statements are used to perform various operations on the data, such as creating tables, inserting data, querying data, updating records, and deleting records.

SQL, which stands for Structured Query Language, is a domain-specific language used to manage and manipulate relational databases. It provides a standardized way to interact with databases, allowing users to perform tasks such as creating, retrieving, updating, and deleting data. SQL is used to define the structure of databases, query the data within them, and manage the data itself. Here are some key aspects of SQL:

i. **Data Definition Language (DDL**): DDL statements are used to define and manage the structure of a database. They include commands to create, modify, and delete database objects like tables, indexes, and views. Common DDL statements include:
   - CREATE DATABASE creates a new database.
   - CREATE TABLE defines a new table's structure.
   - ALTER TABLE modifies an existing table's structure.
   - DROP TABLE removes a table and its data from the database.

ii. Data Manipulation Language (DML): DML statements are used to interact with the data within a database. They include commands to insert, retrieve, update, and delete data. Common DML statements include:
   - SELECT retrieves data from one or more tables.
   - INSERT adds new rows to a table.
   - UPDATE modifies existing data in a table.
   - DELETE removes rows from a table.

iii. **Data Query Language (DQL)**: DQL is a subset of DML focused on querying data. The primary DQL statement is SELECT, which retrieves data from one or more tables based on specified criteria.

iv. **Data Control Language (DCL)**: DCL statements are used to control access to data within the database. They manage user privileges and permissions. Common DCL statements include:
   - GRANT assigns specific privileges to users or roles.

- REVOKE removes privileges from users or roles.

v. **Data Transaction Language (DTL):** DTL statements manage transactions, which are sequences of one or more SQL statements that are treated as a single unit of work. They ensure data consistency and integrity by allowing either all changes to be committed or none at all. Common DTL statements include:
   - COMMIT saves changes made in the current transaction.
   - ROLLBACK undoes changes made in the current transaction.
   - SAVEPOINT creates a point within a transaction to which you can later roll back.

vi. **Joins and Subqueries**: SQL supports the ability to combine data from multiple tables using joins and subqueries. Joins allow you to combine rows from two or more tables based on related columns, while subqueries are nested queries that can be used within larger queries to filter or retrieve data.

vii. **Aggregation and Grouping**: SQL enables the use of aggregate functions like SUM, AVG, COUNT, MIN, and MAX to perform calculations on groups of rows. The GROUP BY clause is used to group rows based on one or more columns.

viii. Indexes and Constraints: SQL allows you to define indexes to enhance data retrieval performance and constraints to maintain data integrity. Constraints include primary keys, foreign keys, unique constraints, and more.

ix. **Views**: A view is a virtual table created by a query. It allows users to retrieve and manipulate data from multiple tables as if it were a single table, without altering the underlying data.

5.3. **Data Types**: MySQL supports various data types, such as integers, strings, dates, and more. Data types define the kind of values that can be stored in a column. For example, "INT" might be used for integer values, "VARCHAR" for variable-length character strings, and "DATE" for date values.

i. Numeric Data Types:

- INT or INTEGER: Integer data type for whole numbers.
- TINYINT: Small integer from -128 to 127 (signed) or 0 to 255 (unsigned).
- SMALLINT: Medium-sized integer from -32,768 to 32,767 (signed) or 0 to 65,535 (unsigned).
- MEDIUMINT: Integer from -8,388,608 to 8,388,607 (signed) or 0 to 16,777,215 (unsigned).
- BIGINT: Large integer from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 (signed) or 0 to 18,446,744,073,709,551,615 (unsigned).
- FLOAT: Single-precision floating-point number.
- DOUBLE or REAL: Double-precision floating-point number.
- DECIMAL or NUMERIC: Exact decimal with specified precision and scale.

ii. **String Data Types:**

- CHAR: Fixed-length character string.
- VARCHAR: Variable-length character string.
- TEXT: Variable-length text data.
- ENUM: Enumeration of possible values.

- SET: Set of values chosen from a predefined list.

iii. **Date and Time Data Types:**

- DATE: Date in 'YYYY-MM-DD' format.
- TIME: Time in 'HH:MM:SS' format.
- DATETIME: Date and time in 'YYYY-MM-DD HH:MM:SS' format.
- TIMESTAMP: Similar to DATETIME but automatically converts to UTC for storage.
- YEAR: Year in 2- or 4-digit format.

iv. **Binary Data Types:**

- BINARY: Fixed-length binary data.
- VARBINARY: Variable-length binary data.
- BLOB: Binary large object, for storing large binary data.
- BIT: A bit-field type representing a series of boolean values.

v. **Spatial Data Types:** MySQL also supports spatial data types for storing geographic data.

- POINT: A single point in space.
- LINESTRING: A sequence of points forming a continuous line.
- POLYGON: A closed loop of line segments forming a polygon.
- GEOMETRY: A generic spatial data type.
- GEOMETRYCOLLECTION: A collection of various geometry objects.

vi. **JSON Data Type:** MySQL supports a JSON data type for storing JSON-formatted data.

vii. **Other Data Types:**

- BOOLEAN or BOOL: Alias for TINYINT(1) used to store true/false values.
- SERIAL: An auto-incrementing integer used as a primary key.

5.4. **Security and User Management**: MySQL provides user authentication and authorization mechanisms to control access to the database. Users can be granted specific privileges, such as SELECT, INSERT, UPDATE, and DELETE, to control what operations they can perform on the data.

Here's an overview of MySQL's security and user management capabilities:

- **User Accounts:** MySQL uses user accounts to authenticate and authorize users. Each user account has a username and associated password. Users must provide valid credentials to access the database.
- **Privileges and Permissions:** MySQL allows administrators to assign specific privileges to user accounts. Privileges control what actions a user can perform within the database. Some common privileges include:
    - SELECT: Ability to retrieve data from tables.
    - INSERT: Ability to add new rows to tables.
    - UPDATE: Ability to modify existing data.
    - DELETE: Ability to remove rows from tables.
    - CREATE: Ability to create new databases and tables.

    - DROP: Ability to delete databases and tables.
    - GRANT: Ability to grant or revoke privileges for other users.
- **Roles**: MySQL supports roles, which are collections of privileges that can be assigned to multiple users. Roles simplify user management by allowing you to define privileges once and then assign the role to multiple users.
- **Authentication Methods**: MySQL supports various authentication methods, including:
    - **MySQL Native Authentication**: Basic username and password authentication.
    - **Pluggable Authentication**: External authentication methods such as LDAP, PAM, or Windows Active Directory.
- **Password Policies**: MySQL allows administrators to set password policies to enforce strong passwords. Password policies can require a minimum password length, a mix of characters, and expiration intervals.
- **SSL/TLS Encryption**: MySQL supports encrypted connections using SSL/TLS protocols. This helps secure data transmission between the client and the server, preventing unauthorized interception.
- **Auditing and Logging:** MySQL provides auditing and logging mechanisms to track user activities and monitor database access. These logs help administrators identify security breaches or unauthorized access.
- **IP Address Whitelisting**: Administrators can configure MySQL to only allow connections from specific IP addresses or ranges, enhancing network-level security.
- **Limiting Connection Sources**: MySQL allows administrators to restrict connections to specific sources, such as localhost, to prevent external access.

5.5. **Triggers and Stored Procedures:** MySQL supports triggers and stored procedures. Triggers are actions that are automatically executed when certain events occur (like an INSERT or UPDATE operation), while stored procedures are precompiled sets of SQL statements that can be executed as a single unit.

5.6. **Backup and Recovery**: Regular backups are crucial for data security. MySQL provides tools and mechanisms to create backups of databases and restore them in case of data loss.

5.7. Connecting to a MySQL database using PHP

To connect to a MySQL database using PHP, you need to use the MySQLi (MySQL Improved) or PDO (PHP Data Objects) extension. Both extensions provide methods to establish a connection, query the database, and retrieve results. Here, I'll show you how to connect to a MySQL database using MySQLi:

i. **Create a MySQL Database:**

Make sure you have PHP and MySQL installed on your server. You can use tools like XAMPP, WAMP, or MAMP for local development, or a web hosting service for online deployment.

Use a tool like phpMyAdmin or command-line MySQL to create a database, along with tables and data, if needed

ii. Connecting to MySQL Database Using MySQLi:

Here's an example of how to connect to a MySQL database using MySQLi in PHP:

```
DEFINE ('DB_USER', 'root');
DEFINE ('DB_PASSWORD', '');
DEFINE ('DB_HOST', 'localhost');
DEFINE ('DB_NAME', 'company');
$dbc = @mysqli_connect(DB_HOST, DB_USER, DB_PASSWORD, DB_NAME);
if (!$dbc) {
  trigger_error ('Could not connect to MySQL: ' . mysqli_connect_error( ) );
  // echo "string";
  } else { // Otherwise, set the encoding:
Echo "Connected to the database successfully";
}
```

iii. Executing SQL Queries:

After establishing the connection, you can execute SQL queries using the $conn object. Here's a simple example of querying data:

```
$query = "select *  from tblcustomer_driver order by phone Asc";
  $result = mysqli_query($dbc, $query);
  if (mysqli_num_rows($result) > 0)
  {
    $output = '';
    while ($row= mysqli_fetch_array($result))
    {
$output .=
"<optionvalue=".$row['id'].">".$row['fullname']."(".$row['phone'].")</option>";
    }
  }
```

iv. **Closing the Connection:**

It's important to close the database connection after you're done using it, especially in long-running scripts or when handling multiple connections.

Both MySQLi and PDO offer similar functionality for database connection and querying. PDO is known for supporting multiple database systems beyond MySQL, while MySQLi is designed specifically for MySQL databases.

5.8. Performing CRUD (Create, Read, Update, Delete) operations with PHP and MySQL

Performing CRUD (Create, Read, Update, Delete) operations with PHP and MySQL involves using PHP to interact with the MySQL database to perform these essential tasks. Here, I'll guide you through examples of each CRUD operation using MySQLi, which is one of the popular ways to connect to MySQL databases in PHP.

i. Create (INSERT):

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {



if (isset ($_POST['item_name']))
{
    $item_name = $_POST['item_name'];
    $item_name = mysqli_real_escape_string($dbc, $item_name);

}


if (isset ($_POST['weight']))
{
    $weight = $_POST['weight'];
    $weight = mysqli_real_escape_string($dbc, $weight);

}
 $q = "INSERT INTO `tblitems`( `item`,`weight) VALUES ('$item_name'
,'$weight')";
$r = mysqli_query($dbc, $q) or trigger_error ("Query $q\n <br\> MySQL Error:"
. mysqli_error($dbc));

        if (mysqli_affected_rows($dbc) == 1)
        {
            echo "<span style='color:green;'>Record Added for " .
$item_name." with weight $weight</span>";
        }
```

```
        else
        {
            echo "<span style='color:green;'>Record Not Added for " .
$item_name."</span>";
        }
    }

}

}
```

ii. **Read (Select)**

```
$query = "select *  from tblcustomer_driver order by phone Asc";
  $result = mysqli_query($dbc, $query);
  if (mysqli_num_rows($result) > 0)
  {
    $output = '';
    while ($row= mysqli_fetch_array($result))
    {
$output .=
"<optionvalue=".$row['id'].">".$row['fullname']."(".$row['phone']).")</option>
";
    }
  }
```

iii. **Update (Update):**

```
$idToUpdate = 1; // Example: the ID of the record to update
$newName = "New Name";
$newEmail = "new@example.com";

$sql = "UPDATE users SET name='$newName', email='$newEmail' WHERE
id=$idToUpdate";
if ($conn->query($sql) === TRUE) {
   echo "Record updated successfully.";
} else {
   echo "Error updating record: " . $conn->error;
}
```

iv. **Delete (Delete):**

```
$idToDelete = 1; // Example: the ID of the record to delete

$sql = "DELETE FROM users WHERE id=$idToDelete";
if ($conn->query($sql) === TRUE) {
   echo "Record deleted successfully.";
```

```
} else {
    echo "Error deleting record: " . $conn->error;
}
```